

Case Study: Pulse

The problem

timestamp,cpu_utilization

```
2014-02-14 14:30:00,0.132
2014-02-14 14:35:00,0.134
2014-02-14 14:40:00,0.134
2014-02-14 14:45:00,0.134
2014-02-14 14:50:00,0.134
2014-02-14 14:55:00,0.134
2014-02-14 15:00:00,0.134
2014-02-14 15:05:00,0.134
2014-02-14 15:10:00,0.066
```

What I told

Look at this data

What is responded (claude):

This is CPU utilization data over a 40-minute window, sampled every 5 minutes. A few things stand out:

- Flat and stable: from 14:30 to 15:00, utilization sits almost perfectly steady around 0.132–0.134 (about 13.2–13.4%).
- Sudden drop: at 15:10, it falls to 0.066 (6.6%) — roughly a 50% drop from the plateau.

That kind of pattern (long flat plateau, then a sharp step down) often signals either a workload change (a process stopping, load being rebalanced elsewhere) or a monitoring gap/restart. With only 9 points it's hard to say which, but the timing (last point) suggests something changed right at the end of your window rather than a gradual decline.

V1 for prompt:

Look at this data. I'm building a monitoring tool for IT managers running on-prem servers, and I want to know if any of this data would trigger an early warning alert.

Response:

Looking at this in the context of an alerting system, here's the honest answer: it depends entirely on what kind of alert you're building.

Threshold-based alert (e.g., "CPU > 80%"): No trigger.

Every value here is low — 6.6% to 13.4%. Nothing comes close to a typical high-utilization warning threshold. If your tool only watches for "server is overloaded," this dataset is silent.

Anomaly/change-point detection: This would fire, and hard.

For 8 straight readings, CPU sits in an extremely tight band (0.132–0.134, essentially flat — standard deviation of ~0.0002). Then the last reading drops to 0.066. That's roughly a 300+ standard deviation move from the established baseline. In plain terms: the server had a rock-steady behavior pattern for 30+ minutes, then something changed abruptly. Statistically, that's about as strong an anomaly signal as you'll ever see, even though the raw number is "low."

Why this matters for on-prem server monitoring specifically:

A sudden *drop* in CPU is often more diagnostically important than a spike, because it can mean:

- A critical process or service crashed/stopped
- The host became unresponsive and the monitoring agent is reporting stale or degraded data
- A scheduled job legitimately finished (benign case)
- Load balancer pulled traffic away from this node

A pure "high CPU" threshold would completely miss all of the concerning scenarios above.

Recommendation for your tool:

Don't rely on static thresholds alone. Add a rolling-baseline comparison (e.g., z-score or percent-change vs. a moving average/std over the last N samples) so a flatline-then-drop pattern like this gets flagged as "unusual behavior," not just checked against a fixed ceiling. You'd want the alert to say something like "CPU deviated sharply from its recent stable baseline" rather than "CPU too high/low."

One caveat: 9 data points is a very small window. I'd want to see whether 6.6% is normal for this server at certain times of day before calling it a true incident — but as an *early warning* candidate for further investigation, yes, this pattern should trip an alert.

V1 Notes:

What changed in the prompt: Added a defined audience ("IT managers running on-prem servers") and a specific goal ("would this trigger an early warning alert")

What actually improved in the output: It stopped just describing the data and started reasoning about it as an alerting decision — it distinguished threshold-based alerts (silent) from anomaly-detection alerts (would fire hard), and connected the drop to real failure scenarios relevant to your product (crashed process, dead monitoring agent, load rebalancing)

What still failed: It's still guessing at what YOUR specific alert logic actually is (you already have a 5-tier system: normal/noticeable/problematic/critical/action-needed) — it doesn't know that, so its recommendation is generic advice, not tailored to your actual design

What you'd try next: Give it real context about your product's existing tiered alert system, so it can react to your specific design instead of inventing its own from scratch

V2 Prompt:

"Look at this data. I'm building a monitoring tool for IT managers running on-prem servers. My alert system has 5 stages: normal, noticeable, problematic, critical, and action-needed — with hard shutdown/reboot only above 90% usage. Based on this data, which stage would this trigger, if any?"

Response:

Based purely on the utilization values against a staged severity system like this, this data stays in "normal" the entire time.

Here's the reasoning:

- Your stages are presumably scaled against the 0–90%+ range (with 90%+ being the hard shutdown ceiling). A typical mapping might look like: normal (0–50%), noticeable (50–70%), problematic (70–80%), critical (80–90%), action-needed/shutdown (90%+) — exact cutoffs are yours to define, but the shape is usually "most of the range is fine, the top slice is dangerous."
- Every reading here — 13.4% down to 6.6% — sits in the bottom ~15% of that entire scale. None of it approaches even the lowest elevated tier, let alone problematic or critical.
- The drop to 6.6% at the end doesn't change this. In a magnitude-based system, a *lower* percentage is never more severe — it just means less load.

So the honest answer: no stage above "normal" fires here, because your system (as described) is threshold-based on absolute usage, and nothing in this data gets anywhere near the danger zone.

One thing worth flagging for your design, though: this is exactly the kind of data where a purely magnitude-based system and an IT manager's intuition might diverge. A human looking at "rock steady for 30 minutes, then drops by half" would want to know *why*, even though the number itself is harmless. If your 5-stage system is meant to catch things like crashed services or dead monitoring agents (not just overload), you may eventually want a separate signal — something like "unexpected pattern change" — that's orthogonal to the 5 usage-based stages, rather than trying to force anomaly detection into a scale built for measuring load severity. But strictly answering your question as asked: normal, no trigger.

V2 Notes:

What changed in the prompt: Added real context — your actual 5-stage alert system and the 90% hard-shutdown rule

What actually improved in the output: It correctly applied YOUR specific system instead of inventing generic advice — it gave a direct, confident answer ("stays in normal the entire time") tied to your exact stages, instead of a vague "it depends" response

What still failed / honest "didn't fully help" moment: This version actually got *less* useful in one way — earlier (Version 1) it caught the anomaly (flat-then-drop pattern) as something worth flagging. Here, by forcing it to strictly answer within your magnitude-based 5-stage system, it correctly says "no alert fires" — which exposes a real gap in your own design (your stages only measure severity of usage, not weird pattern changes), but you almost lose that insight because it's now buried at the end as a footnote instead of the headline

What you'd try next: Ask it to explicitly separate two things: (1) what your current system would do, and (2) what it thinks **SHOULD** happen, so the design gap doesn't get lost as an afterthought

V3 Prompt:

Look at this data. I'm building a monitoring tool for IT managers running on-prem servers. My alert system has 5 stages: normal, noticeable, problematic, critical, and action-needed — with hard shutdown/reboot only above 90% usage. Answer in two clearly separate sections: (1) What my current system would do with this data, and (2) What you think **SHOULD** happen, if different, and why.

Response:

1) What your current system would do with this data

Nothing. Every reading falls between 6.6% and 13.4% CPU utilization — well within the bottom slice of a scale where "action-needed" and hard shutdown live up near 90%+. A magnitude-based, threshold-only system has no mechanism to react to this data at all. It would

sit in "normal" for all 9 readings, including the drop at the end, since a decrease in load is never more severe than the higher values that preceded it.

2) What I think should happen, if different — and why

I don't think this data should trigger any of your 5 usage-severity stages — your system would be correct to stay quiet on *this specific axis*. Low utilization isn't dangerous, and forcing "normal" to fire a noticeable/problematic alert here would just be a false positive that trains IT managers to ignore your tool.

But I'd flag a gap, not a bug: your 5 stages measure *how much* load there is, and this data has a different kind of signal — *how load is behaving over time*. Eight nearly-identical readings (variance of ~0.0002) followed by an abrupt ~50% drop is a strong statistical anomaly, even though the absolute number stays low. That pattern is often diagnostically meaningful on real on-prem hardware — a crashed service, an agent losing visibility into the host, or a node getting pulled from a load balancer.

My suggestion isn't to bend your existing 5 stages to catch this — it's to add a separate, orthogonal signal (e.g., "unusual pattern detected") that runs alongside the usage-severity ladder rather than inside it. That way your core system stays clean and interpretable (X% usage = stage Y), while IT managers still get a heads-up when something changes abruptly, regardless of which direction it moves or where it sits on the severity scale.

V3 Notes:

- What changed in the prompt: Specified output format — forced two clearly separate sections instead of one flowing paragraph
- What actually improved in the output: The valuable insight (the anomaly-detection gap) stopped being a footnote and became its own clear, dedicated section — you can now actually act on it separately from the "current system" answer, instead of having to dig for it
- What still failed: It's still just one hypothetical dataset — it hasn't been asked to actually justify its confidence, or told what "good" looks like from your side, so you can't yet tell if its reasoning is trustworthy or just sounds smart
- What you'd try next: Add quality/verification criteria — ask it to state its confidence and flag any assumptions it's making

V4 Prompt:

Assumptions I'm making

- Your 5 stages (normal → noticeable → problematic → critical → action-needed) are ordered by *magnitude* of usage, with action-needed/shutdown reserved for 90%+ — I'm assuming there are no other stated dimensions (e.g., rate-of-change, duration, anomaly detection) baked into "noticeable" or "problematic," since none were described.

- I'm assuming the exact percentage cutoffs between normal/noticeable/problematic/critical aren't defined yet, or if they are, they're all comfortably above 13.4% — this seems safe since even "noticeable" concern about CPU load typically starts well above where this data sits.
- I'm treating this as a single, isolated server's data with no other context (no other correlated metrics, no known baseline for what's "typical" for this specific box at this time of day).

What I'm not confident about

- Whether 6.6–13.4% is actually low for *this particular server*. For some workloads (e.g., a dedicated batch-processing box expected to run hot), this range might already be unusual in the other direction. I have no baseline to compare against.
- Whether the 15:10 drop reflects a real event or a data/collection artifact (agent hiccup, rounding, etc.) — with only one low point, I can't distinguish a genuine change from noise.
- Whether your system already has anomaly detection elsewhere in the pipeline that I'm not seeing, which would make my suggestion below redundant.

1) What your current system would do with this data

Nothing would fire. All 9 readings (6.6%–13.4%) sit far below any reasonable "noticeable" threshold, let alone problematic, critical, or the 90% shutdown line. The system would classify every point as **normal**.

2) What I think should happen, if different — and why

I agree it should stay in "normal" on the usage-severity axis — nothing here justifies escalating a magnitude-based alert, and doing so would create false positives.

However, there's a separate signal worth considering: 8 readings hold nearly constant (~0.132–0.134, negligible variance), then the 9th drops by roughly half. That's a strong statistical change-point, independent of whether the absolute number is scary. This kind of pattern can indicate a crashed process, a monitoring agent losing accurate visibility, or a load-balancer shift — the kind of thing IT managers often want to know about even when usage is low.

My suggestion: don't fold this into your 5 magnitude-based stages. Instead, add a parallel "pattern change" flag that runs independently, so your core ladder stays clean and predictable while still surfacing behavioral anomalies. Given my uncertainty about this server's normal baseline, I'd treat this as a candidate worth a human glance — not a confirmed incident.

V4 Notes:

- **What changed in the prompt:** Added a requirement to state assumptions and flag low-confidence points before answering
- **What actually improved in the output:** It exposed real blind spots it was silently guessing around earlier — like not knowing if 6.6-13.4% is actually normal for this specific server, and whether the drop is a real event or just a data glitch. This makes the output more honest and lets you (the human) know exactly where to double-check before trusting it
- **What still failed:** The output is getting long now — good for trust, but if an IT manager needed a fast glance at 3am during an actual incident, this much text is friction, not help
- **What you'd try next:** Add a constraint — keep the actionable part short/scannable, push the reasoning into an optional "why" section

V5 Prompt:

Look at this data. I'm building a monitoring tool for IT managers running on-prem servers. My alert system has 5 stages: normal, noticeable, problematic, critical, and action-needed — with hard shutdown/reboot only above 90% usage. Answer in two clearly separate sections: (1) What my current system would do with this data, and (2) What you think SHOULD happen, if different, and why. Before answering, state any assumptions you're making and flag anything you're not confident about. Keep section 1 to one sentence. Keep section 2 to a maximum of 3 short bullet points — an IT manager needs to be able to scan this in under 10 seconds.

Response:

Assumptions: stages are magnitude-based (0–90%+ scale), no other thresholds or anomaly logic already exist elsewhere in your pipeline, and I don't know this server's normal baseline.

Not confident: whether the final drop to 6.6% is a real event or a data/agent artifact — one low point isn't enough to tell.

1) What your system would do: Nothing fires — all readings (6.6%–13.4%) stay in **normal**.

2) What should happen:

- Stay in "normal" on usage severity — nothing here warrants escalation, and forcing it would create false positives.
- The bigger signal is the abrupt ~50% drop after 8 flat readings — worth a separate "pattern change" flag, not a usage-stage bump.
- Treat it as a candidate for a human glance, not a confirmed incident, given the small sample size.

V5 Notes:

- **What changed in the prompt:** Added constraints — hard length limits (1 sentence for section 1, max 3 bullets for section 2)

- What actually improved in the output: It's now genuinely usable in a real alert scenario — an IT manager can scan the whole thing in seconds, and it kept the most important insight (the pattern-change flag) instead of losing it under padding
- What still failed: None obviously — this is the strongest, most production-ready version so far. If anything, pushing further you might ask it to output as literal structured JSON/data instead of prose, if this were feeding into an actual UI rather than being read by a human
- What you'd try next: For an actual dashboard, you might add "output format: structured JSON with severity/confidence/message fields" — but for a human-readable alert, this version is good.

#PERFECT Prompt:

Look at :

```
timestamp,cpu_utilization
2014-02-14 14:30:00,0.132
2014-02-14 14:35:00,0.134
2014-02-14 14:40:00,0.134
2014-02-14 14:45:00,0.134
2014-02-14 14:50:00,0.134
2014-02-14 14:55:00,0.134
2014-02-14 15:00:00,0.134
2014-02-14 15:05:00,0.134
2014-02-14 15:10:00,0.066
```

You are helping design an alerting system for [describe your product's monitoring context — e.g., 'a tool that watches server resource usage for IT managers']. My alert system has these stages: [list your actual stages and thresholds]. Answer in two clearly separate sections: (1) What my current system would do with this data — one sentence only. (2) What you think SHOULD happen, if different, and why — maximum 3 short bullet points, scannable in under 10 seconds. Before answering, briefly state any assumptions you're making and flag anything you're not confident about.